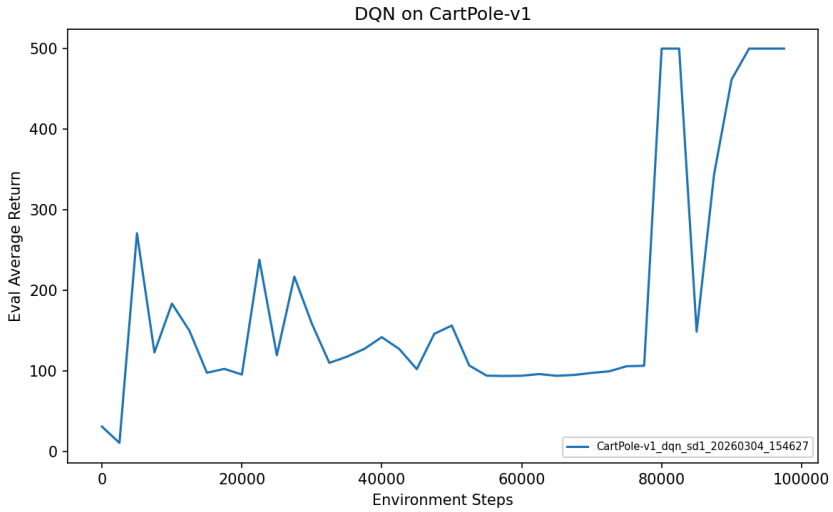
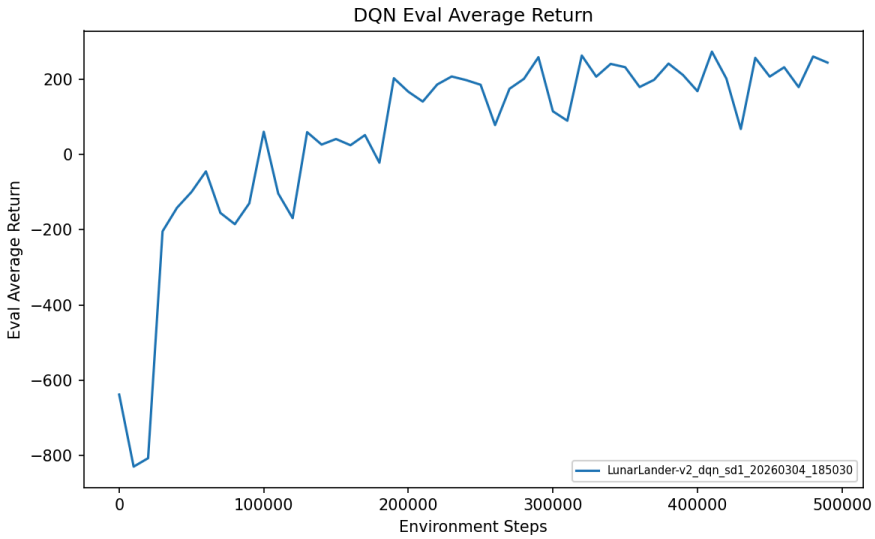
Homework 3 Report

Section 2.4

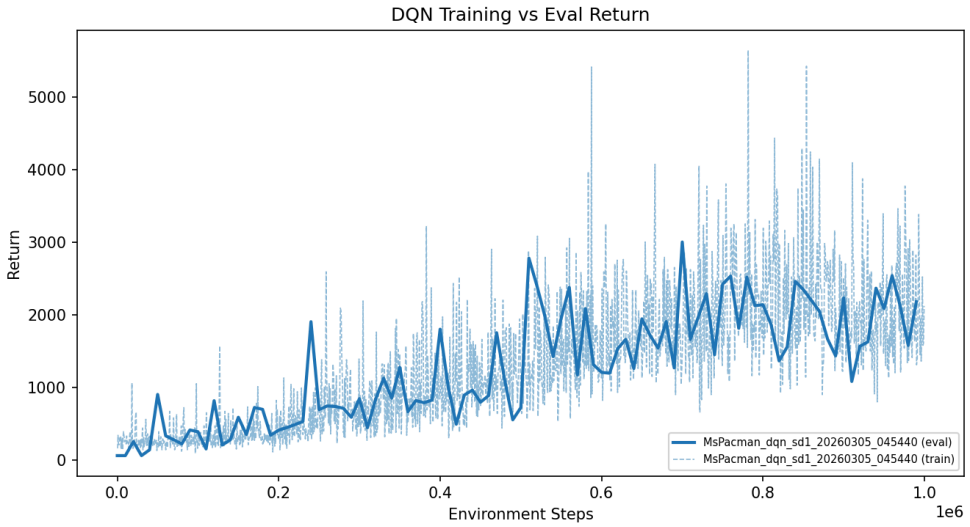


Evaluation return during training for DQN on CartPole-v1 using basic Q-learning algorithm. The x-axis shows the number of environment interaction steps and the y-axis shows the evaluation return. The agent achieves the maximum return of 500

Section 2.5



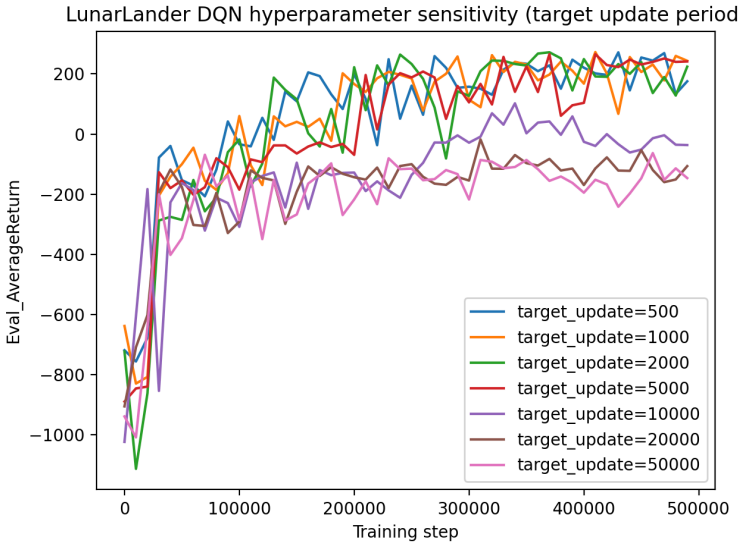
LunarLander-v2 using Double Q-Learning: Eval return over training steps
Evaluation return during training for Double-Q DQN on LunarLander-v2. The x-axis shows environment steps and the y-axis shows evaluation return.



Training return and evaluation return for DQN on MsPacman-v0. The x-axis shows environment steps and the y-axis shows episodic return.

In my results, there's not much difference between training return vs the eval return. Early in training, the agent follows an epsilon-greedy policy so it will perform many random exploratory actions. The evaluation uses greedy policy and averages performance over multiple episodes, so it will have a smoother curve.

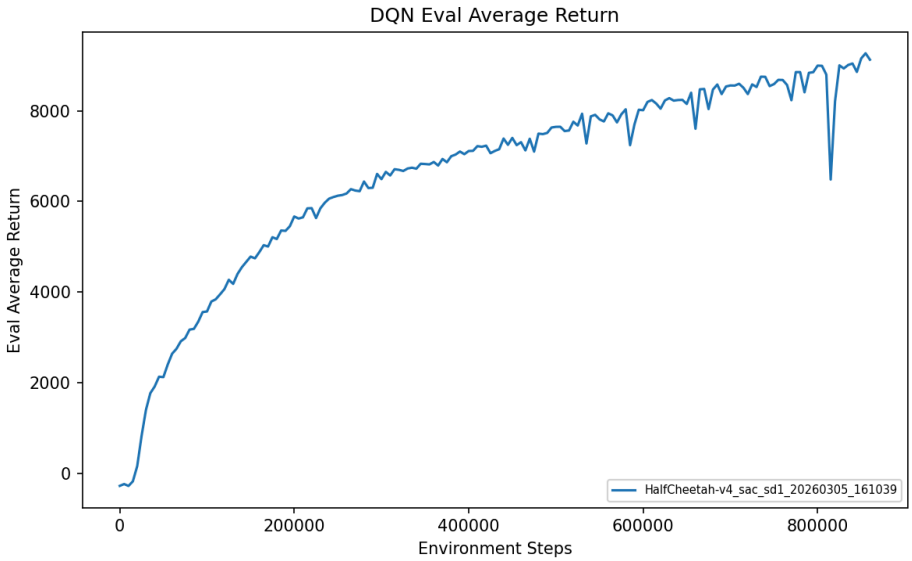
Section 2.6



Sensitivity of DQN performance on LunarLander-v2 to different values of the target network update frequency. The plot shows evaluation return versus environment steps for seven different update frequencies. This experiment illustrates how the choice of target update interval affects training stability and convergence speed.

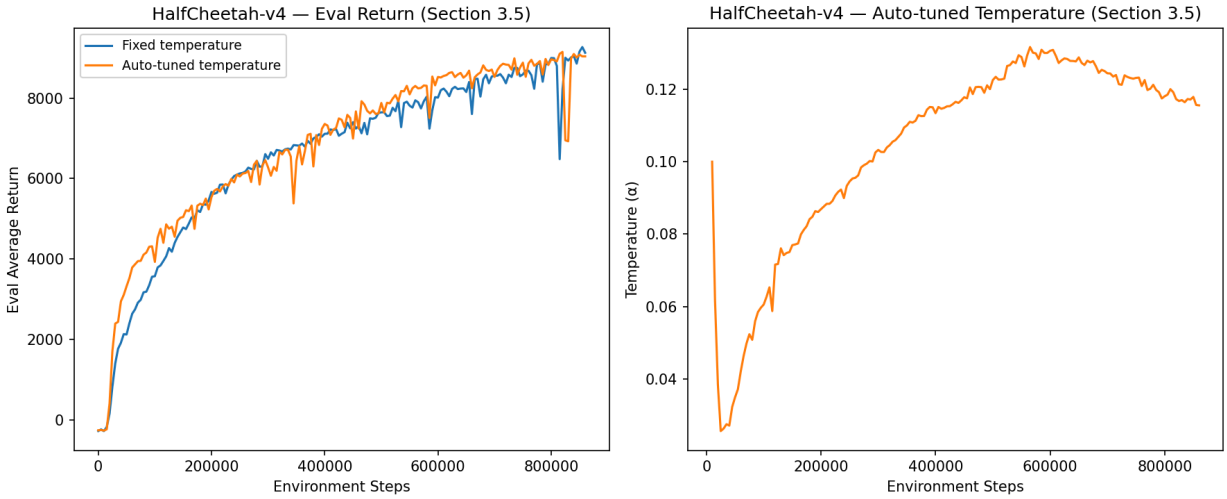
I chose the target network update frequency because it directly affects the stability of Q-learning. I am interested in seeing the trend and effect on eval return on how updating the target network too frequently can make the targets unstable, while updating it too slowly can make learning outdated.

Section 3.4



Evaluation return during training for Soft Actor-Critic on HalfCheetah-v4. The x-axis shows the number of environment steps and the y-axis shows evaluation return. The SAC agent steadily improves performance.

Section 3.5



HalfCheetah-v4 using SAC with Automatic Temperature Tuning: Left subplot is eval return over training for both fixed temperature and auto-tuned temperature. Right subplot is temperature value (α) over training for the auto-tuned run

Does auto-tuning improve or achieve comparable performance to the fixed temperature?

Overall, fixed temperature still has the best performance as it contains the higher max return. However, the auto-tuning achieved comparable performance. Its advantage is that it removes the need for manual hyperparameter tuning and can automatically adapt the temperature to the environment during training.

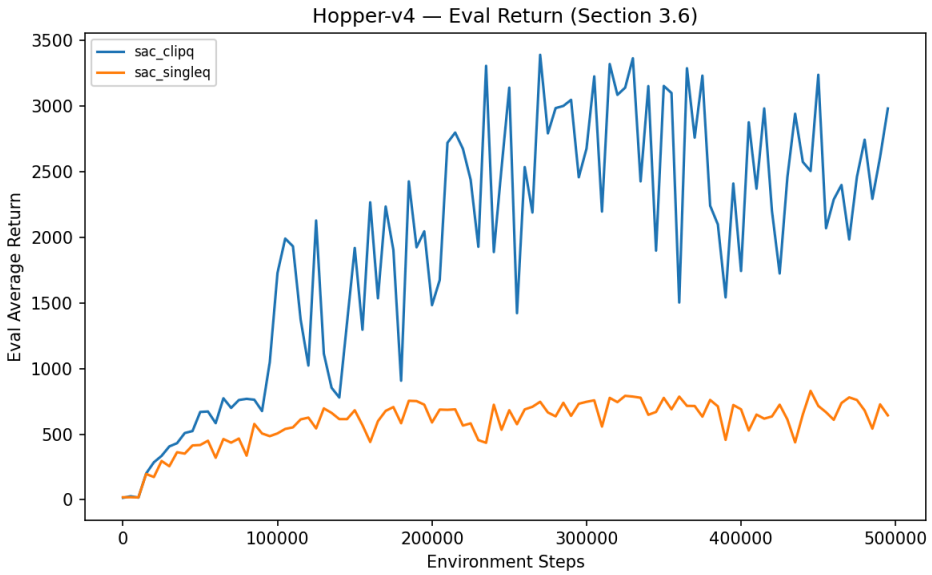
How does the temperature evolve during training? Does it increase, decrease, or stabilize?

Initially, the temperature starts low, then rapidly increases as the algorithm adjusts the entropy level toward the target entropy. Once the policy reaches a suitable exploration level, the temperature stabilizes around 600,000 steps.

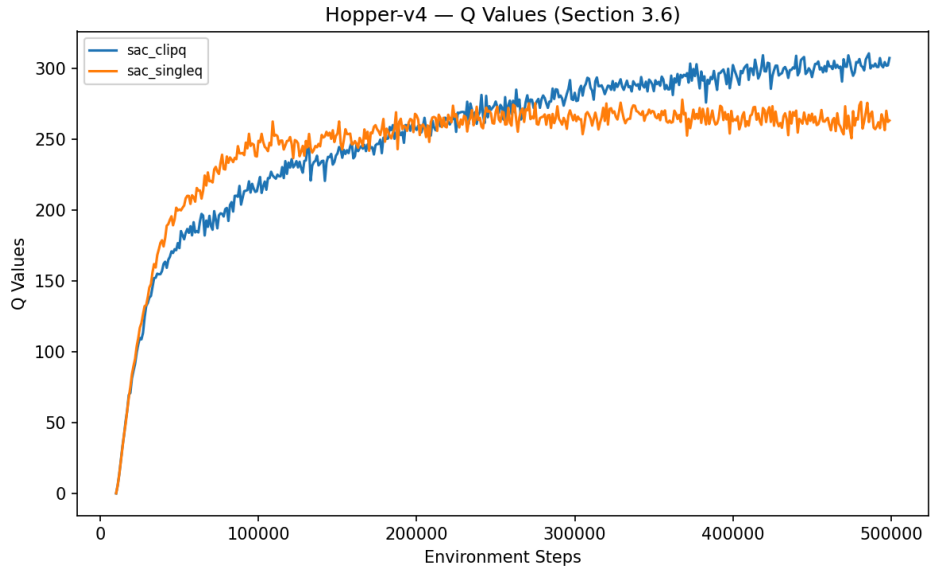
Why might the temperature change in this particular way for HalfCheetah?

Early in training the policy is uncertain and it benefits from higher exploration, which can cause the temperature to increase to encourage higher entropy actions. As the policy improves and begins exploiting good behaviors, the algorithm reduces the temperature so the policy becomes more deterministic and reward-focused. Eventually the temperature stabilizes when the entropy of the policy matches the target entropy.

Section 3.6



Evaluation return during training for SAC on Hopper-v4 comparing single-Q and clipped double-Q critics. The clipped double-Q variant achieves higher returns by reducing overestimation bias in the Q-value targets.



Learned Q-values during training for single-Q and clipped double-Q critics on Hopper-v4.

According to the graphs, the clipped double-Q variant worked better because it achieved higher returns. Also from the Q-values graph, we could clearly see that the clipped double-Q critic is more stable than the single-Q critic.There's a slight instability from 50,000 to 200,000 for the single-Q, where as the double-Q critic steadily increases. The clipped double-Q reduces

overestimation bias in Q-value. The single-Q critic plateaus earlier and is less stable, while clipped double-Q continues improving because it provides more reliable value estimates. Overestimation bias occurs when the max operator in Q-learning selects overly optimistic noisy estimates, and clipped double-Q mitigates this by taking the minimum of two critics when computing the target value. So we could see at the start, the single-Q critic overestimated and exceeded the clipped double-Q.